

Redes neuronais feedforward

Autores

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767



Índice:

1	Introdução	2
2	Preparação dos Dados.....	3
3	Tarefa A – Treino com a Pasta "start"	4
4	Tarefa B – Treino com a Pasta "train"	6
5	Tarefa C – Teste Geral com Todas as Pastas	11
5.1	Classificação das redes da aliena b para a pasta de test	11
5.2	Classificação das redes da aliena b aplicando o treino	12
5.3	Classificação das redes da aliena b usando todas as imagens.....	13
5.4	Melhores redes obtidas.....	15
6	Tarefa D – Testes com Desenhos Próprios.....	16
7	Tarefa E – Aplicação Gráfica em Matlab	19
8	Conclusão Geral	20

Índice de Figuras

Figura 1 - Teste de imagem desenhada kite.....	16
Figura 2 - Teste de imagem desenhada circulo.....	17
Figura 3 - Teste de imagem desenhada quadrado	17
Figura 4 - Teste de imagem desenhada paralelogramo	18
Figura 5 - Menu de criar, treina, guardar e carregar rede.....	19
Figura 6 - Menu de teste de imagens desenhadas pelo utilizador	19

Índice de Tabelas

Tabela 1 – Tabela de redes criadas para o treino da pasta star	4
Tabela 2 - Tabela de redes criadas para o treino com a pasta train	7
Tabela 3 - Melhores redes selecionadas da Tabela 1	8
Tabela 4 - Tabela de comparação das redes não treinadas	11
Tabela 5 - Tabela de comparação das redes treinadas com teste de precisão.....	12
Tabela 6 -Tabela de comparação das redes treinando com todas as imagens	13

1 Introdução

A realização deste trabalho vai permitir aprofundar conhecimentos relativamente a redes neuronais feedforward, na sua aplicação à classificação de figuras geométricas. O trabalho envolve a implementação, treino, avaliação e comparação de diferentes redes neuronais utilizando a ferramenta MATLAB.

Redes neuronais feedforward são um dos modelos utilizados para classificação e regressão dentro da área da inteligência artificial e aprendizagem automática. Estas redes são compostas por camadas de neurónios organizadas sequencialmente, onde a informação flui num sentido único, da camada de entrada até à camada de saída. Cada neurónio aplica uma função de ativação aos dados recebidos, transformando-os antes de os transmitir para a próxima camada.

As redes neuronais feedforward são especialmente eficazes em tarefas de reconhecimento de padrões, como o caso da classificação de imagens. A performance das redes pode ser significativamente influenciada por parâmetros como o número de camadas e neurónios, a escolha das funções de ativação e treino, e a divisão dos dados em conjuntos de treino, validação e teste.

2 Preparação dos Dados

A preparação dos dados, imagens, foi abordada transformando cada imagem numa matriz coluna, através da função *preProcessarImagens(caminho, classes)*, apresentada abaixo, em que caminho corresponde ao caminho onde se encontram as imagens de treino, e classes o tipo de imagens que vão ser analisadas. É fundamental que os nomes definidos em classes estejam de acordo com a organização das subpastas no explorador de ficheiros, pois a função utiliza esses nomes para aceder corretamente às imagens de cada categoria. Cada subpasta deve conter apenas imagens da respetiva classe.

As imagens são lidas por ordem e redimensionadas para um tamanho de 25x25 e de seguida são convertidas para em numeração binária. Cada imagem entra para a matriz de input como um vetor coluna e é adicionado na matriz input, a matriz target corresponde a uma matriz de 6xN, sendo N número de imagens seleccionadas e 6 as diferentes classes de imagens em estudo.

```
function [inputs, targets] = preProcessarImagens(CaminhoPasta, classes)
inputs= [];
targets = []; %Inicializa as matrizes de entrada e saída
imgSize = [25 25]; %Todas as imagens serão redimensionadas para 25x25

%Junta o nome da classe com o caminho da pasta, e lista todos os ficheiros
for i= 1:length(classes)
    path = fullfile(CaminhoPasta, classes{i});
    files = dir(fullfile(path, '*.png'));
    for j = 1:length(files)
        img = imread(fullfile(path, files(j).name)); %lê ficheiro
        if size(img,3) == 3
            img = rgb2gray(img); %Converte para tons cinzentos
        end
        bin = imbinarize(imresize(img, imgSize));
        %Converte a imagem para binario e redimensiona para 32x32
        inputs(:, end+1) = bin(:); %Achata a imagem
        %bin transforma a matriz 2D num vetor coluna
        t = zeros(length(classes),1); %vetor com numero de classes
        t(i) = 1;
        targets(:, end+1) = t;
    end
end
end
```

3 Tarefa A – Treino com a Pasta "start"

Nesta primeira etapa do trabalho, foi realizado o treino de redes neuronais feedforward utilizando exclusivamente as imagens disponíveis na pasta start. Esta pasta contém um número reduzido de imagens (5 por classe), utilizadas tanto como dados de entrada como para treino da própria rede.

Foram testadas várias configurações da rede neuronal, variando o número de neurónios e camadas e as funções de ativação. Contudo, uma vez que os dados de treino e de teste eram exatamente os mesmos, todas as redes alcançaram uma precisão global de 100%, ou seja, um erro de classificação de 0%. Estes resultados eram o esperado, visto que a rede foi treinada com os mesmos exemplos que teve de classificar na sua criação.

Tabela 1 – Tabela de redes criadas para o treino da pasta star

Nº Esc.	Nº Neur.	F. ativação	F. treino	% Global	% Erro
1	10	tansig, purelin	trainlm	100	0
1	20	tansig, purelin	trainlm	100	0
1	10	logsig, satlin	trainlm	100	0
1	20	logsig, satlin	trainlm	100	0
2	10 10	tansig, purelin	trainlm	100	0
2	10 10	logsig, satlin	trainlm	100	0
2	20 20	tansig, purelin	trainlm	100	0
2	20 20	logsig, tansig	trainlm	100	0

Nº. Esc. - Número de Camadas Escondidas

Nº Neur. - Número de Neurónios

F. Ativação - Função de Ativação

F. Treino - Função de Treino

Div. Dados - Divisão Treino/Validação/Teste

% Global – Precisão Global

% Erro – Precisão Erro

De seguida encontra-se o código utilizado para a criação da rede inicial.

```

% Classes e diretório das imagens

classes = {'circle', 'kite', 'parallelogram', 'square', 'trapezoid',
'triangle'};
caminho = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\start';

% Pré-processamento das imagens
[entradas, saidas] = preProcessarImagens(caminho, classes);

% Criação da rede
rede = feedforwardnet([10]); %quantidade de nos
%rede.layers{1}.transferFcn = 'logsig'; %camada de entrada
%rede.layers{2}.transferFcn = 'tansig'; %camada escondida
rede.trainFcn = 'trainlm';
rede.trainParam.epochs = 100;
rede.divideFcn = ''; % Usa todos os dados para treino

% Treinamento
[rede, infoTreino] = train(rede, entradas, saidas);

% Simulação
previsoes = sim(rede, entradas);
disp(class(previsoes));

% Avaliação
erro = perform(rede, saidas, previsoes);
fprintf("Erro Global = %.2f\n", erro);

[~, pred] = max(previsoes);
[~, real] = max(saidas);
precisao = sum(pred == real) / numel(real) * 100;
fprintf("Precisão: %.2f%%\n", precisao);

% Visualização
plotconfusion(saidas, previsoes);
%plotperform(infoTreino);
view(rede);

% Guarda a rede
save('rede_inicial.mat', 'rede');

```

4 Tarefa B – Treino com a Pasta "train"

Nesta tarefa foram criadas 10 configurações de redes, para garantir a fiabilidade dos resultados, a precisão global e de teste de cada configuração apresentada na tabela seguinte foi calculada com base na média de 10 repetições independentes por configuração. Este procedimento permitiu reduzir o impacto de variações aleatórias inerentes ao processo de treino e validação das redes neuronais, assegurando uma avaliação mais robusta e representativa do desempenho de cada topologia testada.

As diferentes funções de treino e ativação utilizadas em redes neuronais apresentam comportamentos distintos que influenciam significativamente o desempenho das redes, tanto em termos de erro total como de precisão. A função `traingd` (Gradient Descent), sendo a mais básica, tende a apresentar desempenhos mais fracos, no entanto a variação do número de camadas e de neurónios, o que justifica os seus valores mais baixos de precisão. Já a função `trainbfg` (Quasi-Newton), embora mais sofisticada e rápida, pode não se adaptar tão bem a todos os tipos de dados, o que leva a uma baixa performance.

Por outro lado, `trainrp` (Resilient Backpropagation) representa uma evolução em termos de robustez. Esta abordagem oferece uma melhor estabilidade e rapidez, mas ainda assim pode mostrar instabilidade em conjuntos de dados muito pequenos. A função `trainlm` (Levenberg-Marquardt) destaca-se como a mais eficaz para redes de pequena e média dimensão, combinando as vantagens de diferentes algoritmos e proporcionando rápida convergência e elevada precisão, sendo frequentemente associada aos melhores resultados em termos de erro e precisão.

Finalmente, `radbasn`, função de ativação usada em redes de base radial (RBF), apresenta um desempenho baixo devido ao uso de poucos neurónios para avaliação. Pode atingir bons resultados mas necessita de mais dados para a configuração utilizada não foi uma boa opção, é uma rede que necessita de grande número de neurónios para apresentar bons resultados.

Em suma, os valores mais baixos observados para algumas destas funções podem ser justificados pelo nível de camadas utilizadas na rede, a capacidade de generalização e de compatibilidade com o problema em estudo.

Tabela 2 - Tabela de redes criadas para o treino com a pasta train

Nº Esc.	Nº Neur.	F. Ativação	F. Treino	Div. Dados	% Global	% Erro
1	5	tansig, purelin	trainlm	0.7/0.15/0.15	78,54	54,44
1	10	tansig, purelin	trainlm	0.7/0.15/0.15	88,53	65,66
2	5 15	tansig, tansig, purelin	trainlm	0.7/0.15/0.15	81,80	54,89
2	10 15	tansig, purelin	traingd	0.7/0.15/0.15	89,40	54,89
1	10	tansig, purelin	traingd	0.7/0.15/0.15	26,67	22,22
1	10	tansig, purelin	trainbfg	0.7/0.15/0.15	31,07	26,22
1	10	tansig, purelin	trainlm	0.7/0.15/0.15	88,87	69,56
1	10	tansig, purelin	trainrp	0.7/0.15/0.15	25,83	20,67
1	10	logsig, purelin	trainlm	0.7/0.15/0.15	84,87	66,33
1	10	tansig, logsig	trainlm	0.7/0.15/0.15	56,90	46,22
1	10	purelin, logsig	trainlm	0.7/0.15/0.15	30,57	27,33
1	10	logsig, tansig	trainlm	0.7/0.15/0.15	89,83	66,00
1	10	radbasn, tansig	trainlm	0.7/0.15/0.15	62,93	37,78
1	10	tansig, purelin	trainlm	0.6/0.2/0.2	86,83	66,83
1	10	tansig, purelin	trainlm	0.8/0.1/0.1	90,03	63,67
1	10	tansig, purelin	trainlm	0.5/0.2/0.3	81,03	60,89

Redes com uma única camada escondida mostraram, no geral, melhor desempenho e maior estabilidade em comparação com redes compostas por duas camadas escondidas. As configurações com duas camadas (por exemplo, 5-15 neurónios) não apresentaram melhorias significativas e, em alguns casos, registaram uma diminuição da precisão global e um erro relativamente elevado. Para este problema específico, conclui-se que arquiteturas mais simples, com apenas uma camada escondida, foram suficientes e mais eficazes.

Relativamente ao número de neurónios, observou-se que aumentar de 5 para 10 neurónios numa camada escondida resultou, em média, numa melhoria da precisão global (de 78,54% para 88,53%) e numa redução do erro. No entanto, redes com mais neurónios ou com múltiplas camadas (como a configuração 10-15) não demonstraram ganhos justificativos, apresentando resultados semelhantes à configuração com 10 neurónios numa só camada. Assim, a utilização de

10 neurónios numa única camada revelou-se uma solução com bom desempenho e menor complexidade.

Comparando as funções de ativação, a combinação tansig na camada escondida com purelin na camada de saída foi claramente a mais eficaz. Esta configuração foi a que, apresentou as maiores taxas de acerto, sobretudo quando utilizada em conjunto com a função de treino trainlm.

Por fim analisando a variação da divisão dos dados entre treino, validação e teste, verificou-se uma tendência geral de melhoria da taxa de precisão global quando a percentagem atribuída ao conjunto de treino era maior. No entanto, este aumento nem sempre se refletiu diretamente numa melhor rede. Por exemplo, apesar de a divisão 0.6/0.2/0.2 ter um menor volume de dados de treino comparada com a divisão 0.8/0.1/0.1, obteve melhores resultados de teste em algumas configurações. Isto sugere que, além da quantidade, a qualidade da generalização do modelo é um fator essencial, devendo a divisão de dados ser ajustada com base no comportamento observado.

Tabela 3 - Melhores redes seleccionadas da Tabela 1

Nº Esc.	Nº Neur.	F. Ativação	F. Treino	Div. Dados	% Global	% Erro
1	10	tansig, purelin	trainlm	0.7/0.15/0.15	88,87	69,56
1	10	tansig, purelin	trainlm	0.6/0.2/0.2	86,83	66,83
1	10	logsig, purelin	trainlm	0.7/0.15/0.15	84,87	66,33

Para todas as configurações os dados foram registados no ficheiro “b – dados obtidos.txt”.

```
% Definir classes e caminho
```

```
classes = {'circle', 'kite', 'parallelogram', 'square', 'trapezoid',  
          'triangle'};
```

```
CaminhoPasta = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º  
Semestre\CR\TP\ImageFolders\train';
```

```
% Pré-processar imagens
```

```
[inputs, targets] = preProcessarImagens(CaminhoPasta, classes);
```

% Configurações dos testes

```
configs = {
    {'Conf1', 5, {'tansig', 'purelin'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf2', 10, {'tansig', 'purelin'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf3', [5 15], {'tansig', 'tansig', 'purelin'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf4', [10 15], {'tansig', 'tansig', 'purelin'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf5', [10], {'tansig', 'purelin'}, 'traingd', [0.7 0.15 0.15]},
    {'Conf6', [10], {'tansig', 'purelin'}, 'trainbfg', [0.7 0.15 0.15]},
    {'Conf7', [10], {'tansig', 'purelin'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf8', [10], {'tansig', 'purelin'}, 'trainrp', [0.7 0.15 0.15]},
    {'Conf9', [10], {'logsig', 'purelin'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf10', [10], {'tansig', 'logsig'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf11', [10], {'purelin', 'logsig'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf12', [10], {'logsig', 'tansig'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf13', [10], {'radbasn', 'tansig'}, 'trainlm', [0.7 0.15 0.15]},
    {'Conf14', [10], {'tansig', 'purelin'}, 'trainlm', [0.6 0.2 0.2]},
    {'Conf15', [10], {'tansig', 'purelin'}, 'trainlm', [0.8 0.1 0.1]},
    {'Conf16', [10], {'tansig', 'purelin'}, 'trainlm', [0.5 0.2 0.3]}
};
```

% Armazenar resultados e redes

```
resultados = {};
todasRedes = {};
```

% Testar cada configuração

```
for i = 1:length(configs)
    conf = configs{i};
    nomeConf = conf{1};
    topologia = conf{2};
    funcoesAtivacao = conf{3};
    funcaoTreinamento = conf{4};
    divisao = conf{5};

    % Exibir configurações
    fprintf('Configuração: %s\n', nomeConf);
    fprintf('Topologia: %s\n', mat2str(topologia));
    fprintf('Funções de Ativação: %s\n', strjoin(funcoesAtivacao, ' '));
    fprintf('Função de Treinamento: %s\n', funcaoTreinamento);
    fprintf('Divisão: %.2f / %.2f / %.2f\n', divisao(1), divisao(2), divisao(3));
```

% Inicializar variáveis para armazenar os resultados das 10 repetições

```
erroTotal = 0;
accGlobalTotal = 0;
accTesteTotal = 0;
```

% Realizar 10 repetições para cada configuração

```
for rep = 1:10
    % Criar e configurar rede
    rede = feedforwardnet(topologia);
    rede.trainFcn = funcaoTreinamento;
    rede.trainParam.epochs = 100;
    rede.divideParam.trainRatio = divisao(1);
    rede.divideParam.valRatio = divisao(2);
    rede.divideParam.testRatio = divisao(3);
```

```

    % Definir funções de ativação
    for j = 1:length(rede.layers)
        rede.layers{j}.transferFcn = funcoesAtivacao{min(j,
length(funcoesAtivacao))};
    end

    % Treinar rede
    [rede, tr] = train(rede, inputs, targets);

    % Simulação
    outputs = sim(rede, inputs);

    % Calcular métricas
    erro = perform(rede, targets, outputs);
    [~, pred] = max(outputs);
    [~, real] = max(targets);
    accGlobal = sum(pred == real) / length(real) * 100;

    % Precisão do teste
    outTeste = outputs(:, tr.testInd);
    tTeste = targets(:, tr.testInd);
    [~, predTeste] = max(outTeste);
    [~, realTeste] = max(tTeste);
    accTeste = sum(predTeste == realTeste) / length(realTeste) * 100;

    % Acumular os resultados das repetições
    erroTotal = erroTotal + erro;
    accGlobalTotal = accGlobalTotal + accGlobal;
    accTesteTotal = accTesteTotal + accTeste;
end

% Calcular as médias dos resultados das 10 repetições
erroMedio = erroTotal / 10;
accGlobalMedio = accGlobalTotal / 10;
accTesteMedio = accTesteTotal / 10;

% Armazenar os resultados médios e redes
resultados = [resultados; {nomeConf, erroMedio, accGlobalMedio, accTesteMedio}];
todasRedes{i}.nome = nomeConf;
todasRedes{i}.net = rede;
todasRedes{i}.accTeste = accTesteMedio;

% Exibir os resultados médios
fprintf('Média do Erro: %.8f\n', erroMedio);
fprintf('Média da Precisão Global: %.2f%%\n', accGlobalMedio);
fprintf('Média da Precisão de Teste: %.2f%%\n\n', accTesteMedio);
end

% Ordenar e salvar as melhores redes atraves de accTesteMedio
[~, idxOrdenado] = sort(cell2mat(resultados(:,4)), 'descend');
for k = 1:min(3, length(idxOrdenado))
    rede = todasRedes{idxOrdenado(k)}.net;
    nomeFicheiro = sprintf('bestB%d.mat', k);
    save(nomeFicheiro, 'rede');
end

```

5 Tarefa C – Teste Geral com Todas as Pastas

5.1 Classificação das redes da aliena b para a pasta de test

A rede bestB1.mat foi a rede mais eficaz entre as três quando aplicada aos dados de teste, com a maior precisão e o menor erro. Isso sugere que, provavelmente, esta rede tem melhor generalização.

Tabela 4 - Tabela de comparação das redes não treinadas

	Erro Total	Precisão Global
bestB1.mat	0.08283630	75.00%
bestB2.mat	0.16812286	66.66%
bestB3.mat	0.11774881	60.00%

```
% Definir classes e caminho

classes = {'circle', 'kite', 'parallelogram', 'square', 'trapezoid',
'triangle'};
pastaImagens = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\test';

% Pré-processamento das imagens
[entradas, saidas] = preProcessarImagens(pastaImagens, classes);
% Carregar as melhores redes de b
%load('bestB1.mat');
load('bestB2.mat');
%load('bestB3.mat');
% Simulação da rede e cálculo de erro
previsoes = sim(rede, entradas);
erroTotal = perform(rede, saidas, previsoes);
fprintf("Erro Total = %.8f\n", erroTotal);

% Cálculo da precisão teste
[~, previsaoClasses] = max(previsoes);
[~, classesReais] = max(saidas);
precisaoGlobal = sum(previsaoClasses == classesReais) / length(classesReais) *
100;
fprintf("Precisão Global: %.8f%%\n", precisaoGlobal);

% Matriz de confusão para o teste
plotconfusion(saidas, previsoes);

% Visualizar a estrutura da rede
view(rede);
```

5.2 Classificação das redes da aliena b aplicando o treino

Todas as redes mostraram um aumento significativo na precisão de treino e teste quando comparadas com a fase inicial. Isso confirma que o treino de redes neurais com mais dados leva a uma melhoria na precisão global e a uma redução do erro. A rede bestB2.mat apresentou melhor desempenho em termos de precisão global. Podemos ainda reparar que uma rede com melhor precisão de treino pode ser mais fraca quando apresentados novos dados desconhecidos como é o caso da rede bestB1.mat. A rede bestB3.mat acaba por ser uma rede bastante equilibrando tenho bons resultados em quase todos os campos.

Tabela 5 - Tabela de comparação das redes treinadas com teste de precisão

	Precisão Teste	Precisão Global	Precisão Start	Precisão Train	Precisão Test
bestB1.mat	77.77%	75.00%	60.00%	93.33%	66.66%
bestB2.mat	66.66%	88.33%	60.00%	66.66%	77.77%
bestB3.mat	44.44%	85.00%	20.00%	77.77%	88.88%

```
function TreinarETestiar_C_ii(nomeRede)
    % Definição das classes e dos caminhos das pastas de dados
    classes = {'circle', 'kite', 'parallelogram', 'square', 'trapezoid',
    'triangle'};
    caminhoPastaTest = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\test';
    caminhoPastaStart = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\start';
    caminhoPastaTrain = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\train';

    % Processando as imagens da pasta de teste
    [entradas, saidas] = preProcessarImagens(caminhoPastaTest, classes);

    % Carregando a rede previamente treinada de b)
    load(nomeRede);

    % Verificando se a variável 'rede' foi carregada corretamente
    if ~exist('rede', 'var')
        error('A rede "rede" não foi carregada corretamente do arquivo "%s".',
nomeRede);
    end

    % Garantindo que a rede é do tipo esperado "network"
    if ~isa(rede, 'network')
        error('A variável "rede" não é uma rede válida do tipo "network."');
    end
```

```

% Calculando a precisão geral do modelo
[~, previsao] = max(saidasPrevistas);
[~, real] = max(saidas);
precisaoGlobal = sum(previsao == real) / length(real) * 100;

% Calculando a precisão no conjunto de teste específico
saidasTeste = saidasPrevistas(:, tr.testInd);
saidasReaisTeste = saidas(:, tr.testInd);
[~, previsaoTeste] = max(saidasTeste);
[~, reaisTeste] = max(saidasReaisTeste);
precisaoTeste = sum(previsaoTeste == reaisTeste) / length(reaisTeste) *
100;

% Exibindo as precisões calculadas
fprintf("Precisão de Teste: %.8f%%\n", precisaoTeste);
fprintf("Precisão Global: %.8f%%\n", precisaoGlobal);

% Avalia o desempenho da rede nas pastas Start, Train e Test
avaliarPasta(rede, caminhoPastaStart, classes, 'Start');
avaliarPasta(rede, caminhoPastaTrain, classes, 'Train');
avaliarPasta(rede, caminhoPastaTest, classes, 'Test');

% Gerando a matriz de confusão para os resultados do conjunto de teste
plotconfusion(saidasReaisTeste, saidasTeste);

% Visualizando a estrutura interna da rede neural
view(rede);
end

```

5.3 Classificação das redes da aliena b usando todas as imagens

Ao usar todas as imagens, as redes melhoraram seu desempenho. A rede bestB1.mat continuou a ser a mais forte, com uma precisão global elevada, a par da rede bestB2.mat. Já a bestB3.mat teve o pior desempenho global, mas ainda foi capaz de melhorar seu desempenho de treino e teste.

Tabela 6 -Tabela de comparação das redes treinando com todas as imagens

	Precisão Teste	Precisão Global	Precisão Start	Precisão Train	Precisão Test
bestB1.mat	84.75%	93.84%	100%	95.55%	77.77%
bestB2.mat	76.92%	90.26%	80.00%	93.33%	77.77%
bestB3.mat	76.27%	83.08%	80.00%	88.88%	77.77%

```

function [accTeste, accGlobal, rede, saidasTotal, saidasPrevistas] =
TreinarETestear_c_iii(nomeRede)
    % Função que recebe uma rede de b), treina e testa usando as pastas Start, Train
e Test.
    % Retorna as precisões de teste e global da rede, além da própria rede para
armazenamento.
    disp(nomeRede);
    classes = {'circle', 'kite', 'parallelogram', 'square', 'trapezoid', 'triangle'};
    caminhoPastaTest = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\test';
    caminhoPastaStart = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\start';
    caminhoPastaTrain = 'C:\Users\tiago\OneDrive - ISEC\Informática\2 ano\2º
Semestre\CR\TP\ImageFolders\train';

    % Pré-processando as imagens de todas as pastas (Start, Train e Test)
    [entradasStart, saidasStart] = preProcessarImagens(caminhoPastaStart, classes);
    [entradasTrain, saidasTrain] = preProcessarImagens(caminhoPastaTrain, classes);
    [entradasTest, saidasTest] = preProcessarImagens(caminhoPastaTest, classes);

    % Unindo os dados de entrada e saída para todas as pastas
    entradasTotal = [entradasStart, entradasTrain, entradasTest];
    saidasTotal = [saidasStart, saidasTrain, saidasTest];

    % Carregando a rede já treinada de b), utilizando o nome 'rede'
    load(nomeRede, 'rede'); % Carrega a rede com o nome 'rede'

    % Verificando se a rede foi carregada corretamente
    if ~exist('rede', 'var')
        error('A variável "rede" não foi encontrada no arquivo "%s".', nomeRede);
    end

    % Realizando o treinamento da rede com todos os dados de entrada
    [rede, tr] = train(rede, entradasTotal, saidasTotal);

    % Realizando a simulação da rede usando os dados de entrada
    saidasPrevistas = sim(rede, entradasTotal);

    % Calculando a precisão global considerando todos os dados
    [~, pred] = max(saidasPrevistas);
    [~, real] = max(saidasTotal);
    accGlobal = sum(pred == real) / length(real) * 100;

    % Calculando a precisão específica para o conjunto de teste
    saidasTeste = saidasPrevistas(:, tr.testInd);
    saidasReaisTeste = saidasTotal(:, tr.testInd);
    [~, predTeste] = max(saidasTeste);
    [~, realTeste] = max(saidasReaisTeste);
    accTeste = sum(predTeste == realTeste) / length(realTeste) * 100;

    % Exibindo as precisões de teste e global
    fprintf("Precisão de Teste: %.8f%%\n", accTeste);
    fprintf("Precisão Global: %.8f%%\n", accGlobal);

    % Avaliando a rede nas pastas Start, Train e Test
    avaliarPasta(rede, caminhoPastaStart, classes, 'Start');
    avaliarPasta(rede, caminhoPastaTrain, classes, 'Train');
    avaliarPasta(rede, caminhoPastaTest, classes, 'Test');

    % (Opcional) Gerar a matriz de confusão para os resultados
    plotconfusion(saidasTotal, saidasPrevistas);

```


5.4 Melhores redes obtidas

Para a obtenção das melhores redes da alínea c, foi desenvolvido um código matlab que avalia o desempenho das redes resultantes da alínea b após o treino realizado na alínea iii. O critério de seleção adotado foi a maior precisão global obtida durante os testes.

O algoritmo percorre cada uma das redes geradas a partir do treino aplicado e, para cada caso (correspondente às redes bestB1, bestB2 e bestB3), e guarda a rede que obtenha a melhor taxa de acerto global. Estas redes são então salvas individualmente em ficheiros .mat com os nomes bestC1.mat, bestC2.mat e bestC3.mat, respetivamente.

Este processo permite garantir que, para cada configuração testada na alínea b, apenas a versão mais eficiente e generalizável da rede é considerada para avaliação final e aplicação prática. Para todas as configurações os dados foram registados nos ficheiros “civbx – teste para obter a melhor.txt”, em que x é referente a melhor rede na alínea b.

```
%Recebe uma rede previamente guardada, corre a função TrainETeste_c_iii x vezes e
guarda a melhor rede das x treinadas
%Esta função utiliza as redes guardadas em b. Se se utiliza Bx guarda no ficheiro
Cx.

bestAccTest = 0;
bestAccGlobal = 0;

%Qual das redes de b) se quer fazer load
%nomeRede = 'bestB1.mat';
%nomeRede = 'bestB2.mat';
nomeRede = 'bestB3.mat';

for i = 1 : 10
    fprintf("\nIteração %d:\n", i);
    [accTesteAtual, accGlobalAtual, rede, saidasTemp, PrevistasTemp] = ciiv(nomeRede);
    if accTesteAtual >= bestAccTest && accGlobalAtual >= bestAccGlobal
        bestAccGlobal = accGlobalAtual;
        bestAccTest = accTesteAtual;
        bestRede = rede;
        saidasTotal = saidasTemp;
        saidasPrevistas = PrevistasTemp;
    end
end
fprintf("\n\nMelhor das redes:");
fprintf("Precisão de Teste: %.8f%%\n", bestAccTest);
fprintf("Precisão Global: %.8f%%\n", bestAccGlobal);
rede = bestRede;

% (Opcional) Gerar a matriz de confusão para os resultados
plotconfusion(saidasTotal, saidasPrevistas);

%Guardar a melhor das redes treinadas em b
%save('bestC1.mat', "rede");
%save('bestC2.mat', "rede");
save('bestC3.mat', "rede");
```

6 Tarefa D – Testes com Desenhos Próprios

Nesta etapa, é possível verificar que imagens não centradas ou demasiado próximas das margens do enquadramento podem levar a classificações incorretas, uma vez que a rede pode não conseguir extrair corretamente as características principais da figura. Adicionalmente, linhas mais tremidas ou imprecisas nas imagens desenhadas também prejudicam a identificação correta das formas.

Outro fator crítico é a semelhança entre certas classes geométricas, como os trapézios e paralelogramas, ou ainda kites e triângulos, cujas diferenças podem ser subtis para a rede, especialmente se as imagens não forem perfeitamente desenhadas.

Por fim, o próprio processo de treino das redes pode introduzir enviesamentos, sobretudo se houver um desbalanceamento no número de imagens por classe, o que pode fazer com que a rede favoreça a classificação de formas mais representadas nos dados de treino.

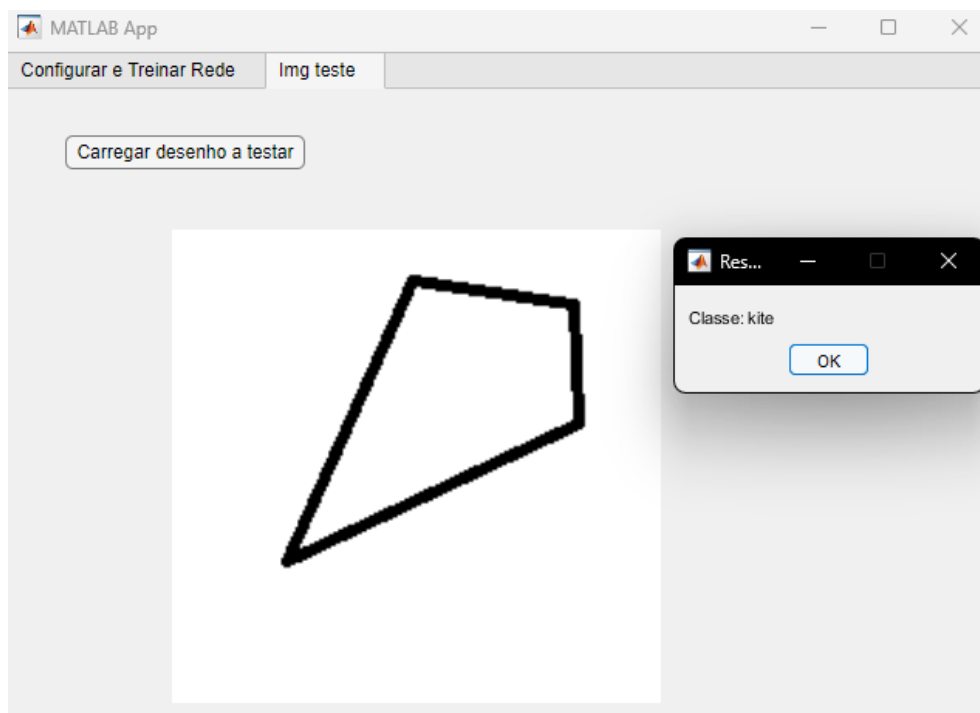


Figura 1 - Teste de imagem desenhada kite

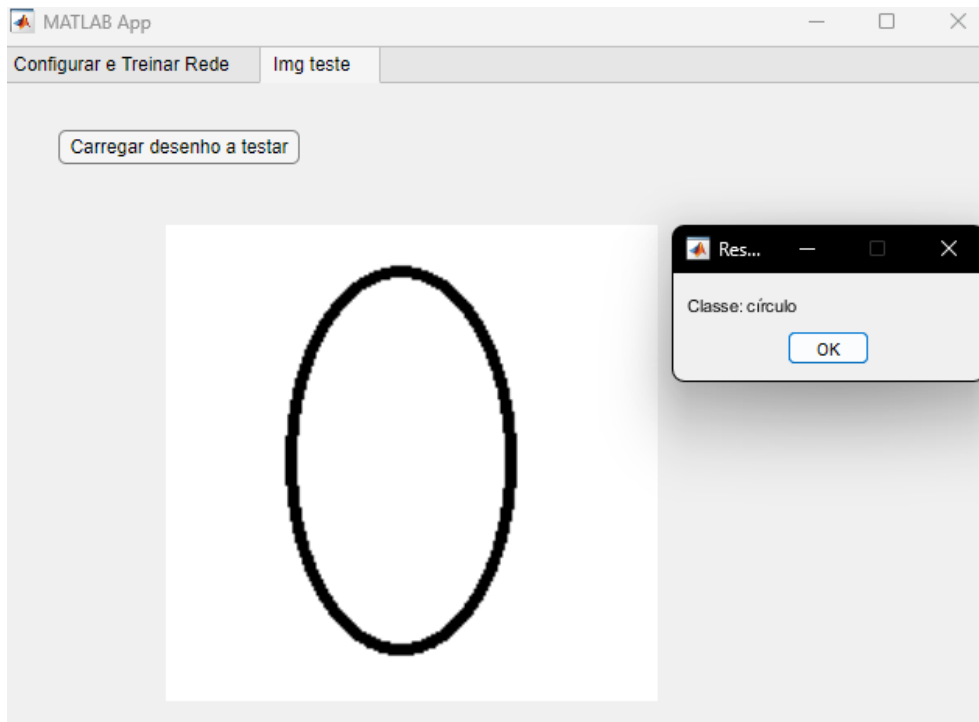


Figura 2 - Teste de imagem desenhada círculo

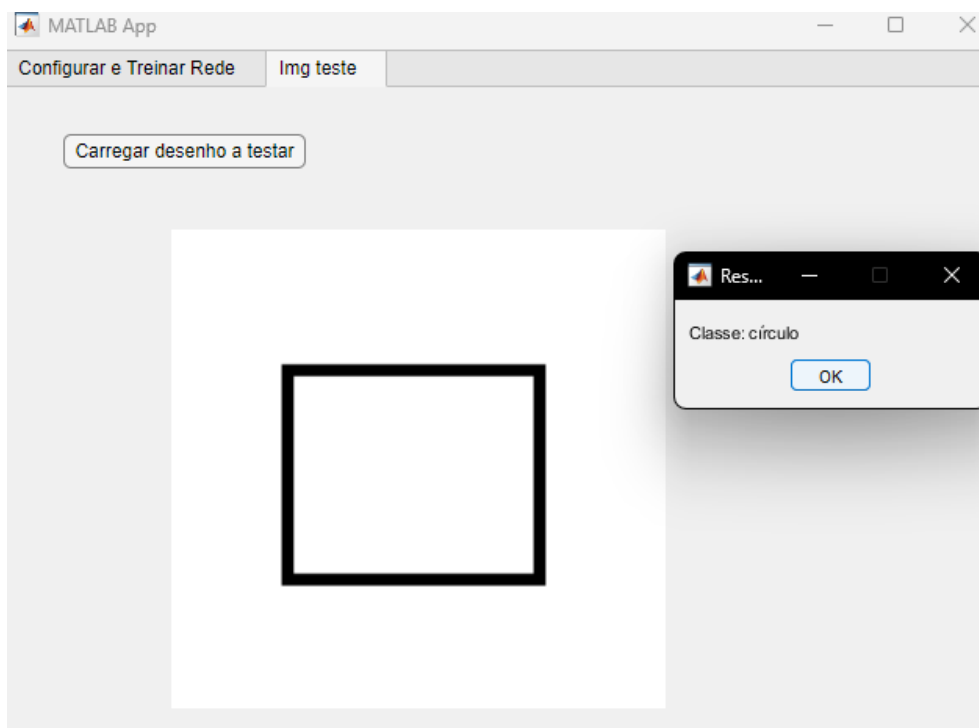


Figura 3 - Teste de imagem desenhada quadrado

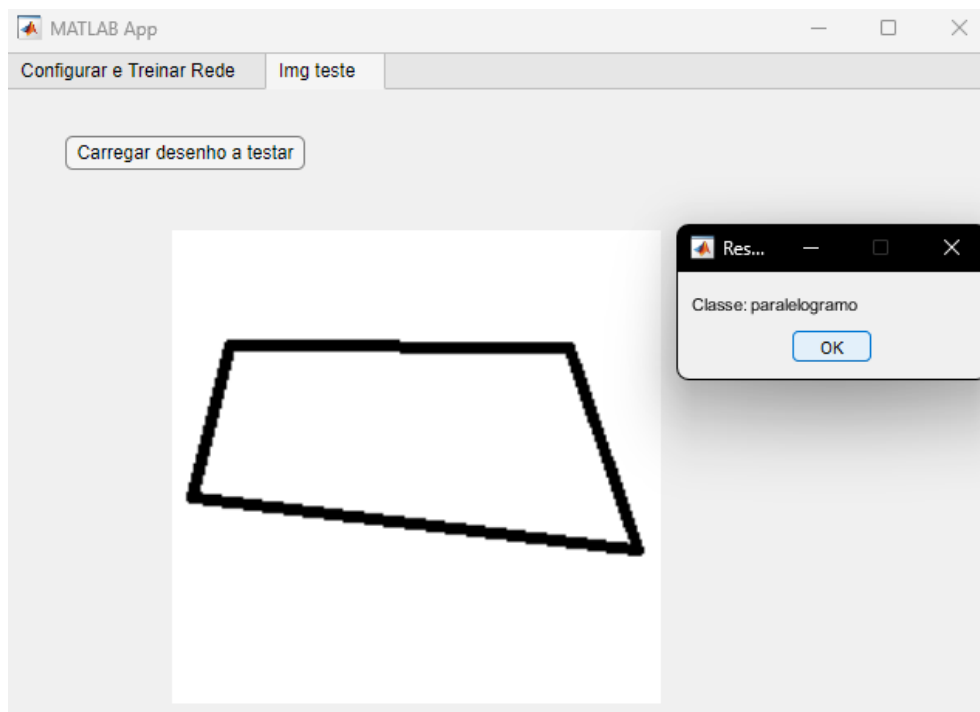


Figura 4 - Teste de imagem desenhada paralelogramo

7 Tarefa E – Aplicação Gráfica em Matlab

Foi desenvolvida uma aplicação gráfica simples em MATLAB que permite ao utilizador criar e manipular uma rede neuronal do tipo feedforward de forma intuitiva. Através desta interface, o utilizador pode definir o número de camadas ocultas, o número de neurónios por camada, seleccionar a função de ativação, bem como a função de treino a utilizar. Após a definição dos parâmetros, a aplicação gera automaticamente a rede neuronal, que pode ser treinada com dados previamente definidos, gravada em ficheiro para uso posterior ou carregada para continuar o treino ou realizar testes.

Adicionalmente, a aplicação possui uma aba que permite ao utilizador carregar uma imagem desenhada manualmente, que é então processada e classificada pela rede carregada. Esta funcionalidade torna possível verificar, de forma imediata, o desempenho da rede na identificação de formas geométricas com base em imagens desenhadas diretamente pelo utilizador.

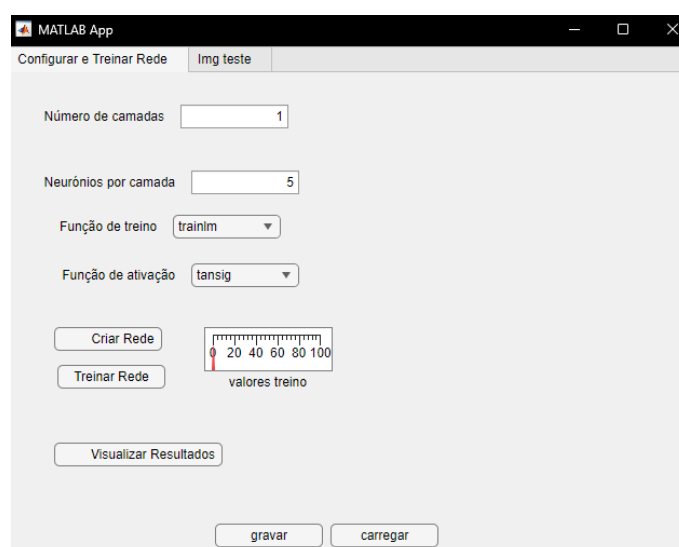


Figura 5 - Menu de criar, treina, guardar e carregar rede

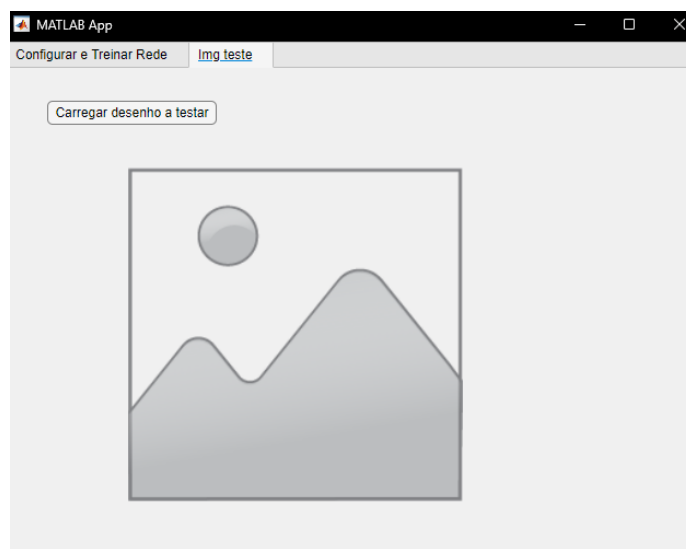


Figura 6 - Menu de teste de imagens desenhadas pelo utilizador

8 Conclusão Geral

Em resumo, pode-se afirmar que o desenvolvimento e a experimentação com redes neuronais para a classificação de formas geométricas permitiram compreender a importância da qualidade e diversidade dos dados de treino, bem como da escolha adequada da arquitetura da rede. Verificou-se que pequenas variações nas imagens, como deslocamento, espessura das linhas ou formas pouco centradas, podem influenciar negativamente a classificação, evidenciando a sensibilidade das redes a dados fora do padrão aprendido. A semelhança entre algumas classes, como trapézios e paralelogramos ou kite e triângulo, também contribuiu para a ocorrência de erros, realçando os limites da capacidade de generalização da rede quando treinada com conjuntos desequilibrados ou com poucas amostras.

A aplicação gráfica desenvolvida demonstrou ser uma ferramenta eficaz para criar, treinar e testar redes de forma simples e interativa, permitindo ainda a classificação de imagens desenhadas manualmente, o que reforça a utilidade prática do sistema implementado.



**Instituto Superior
de Engenharia**

Politécnico de Coimbra